MORPHE

Memory-Oriented Reconfigurable Plasticity and Hierarchical Evolution

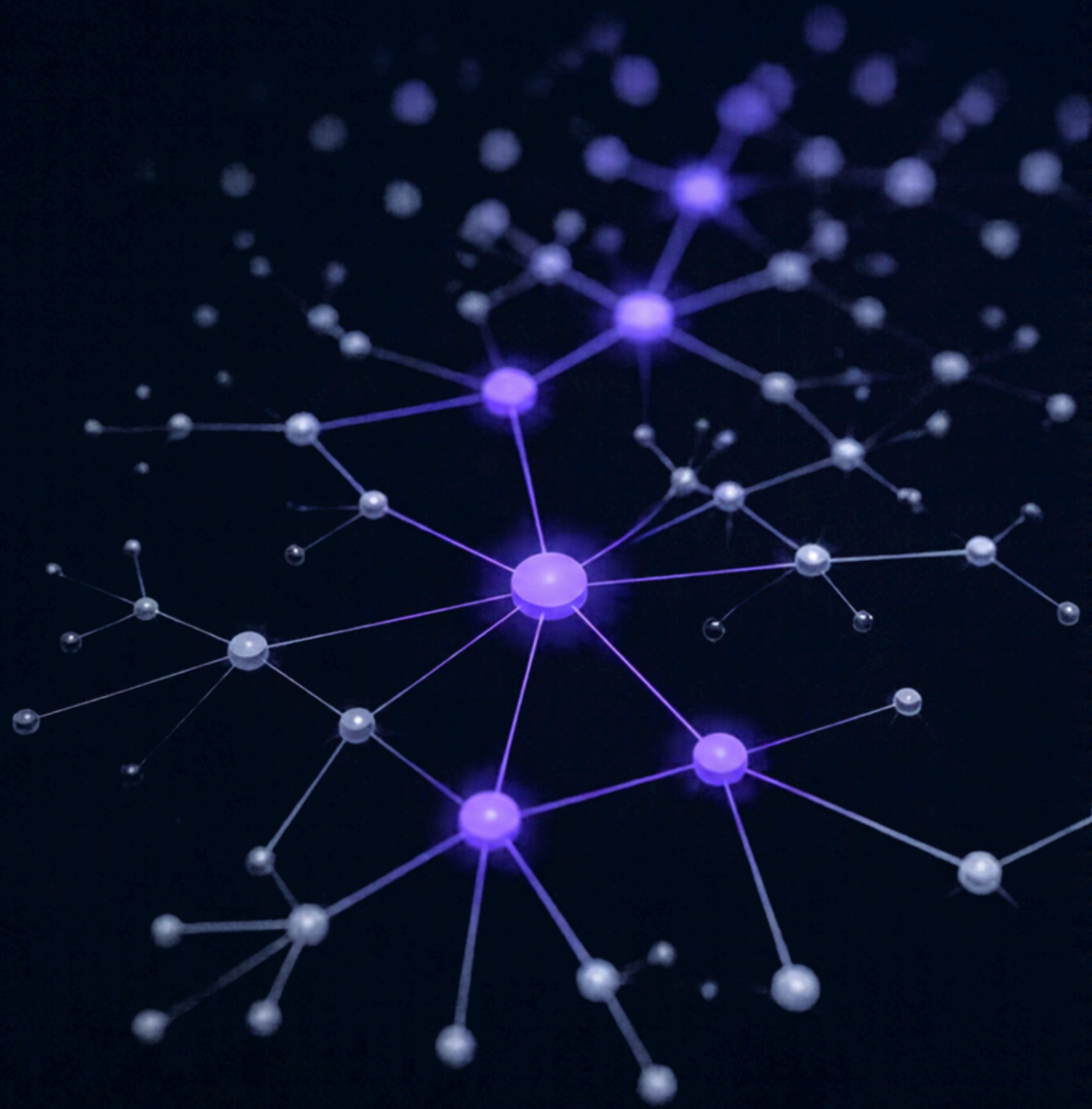
Memory that adapts its structure as retrieval needs evolve.

MORPHE is an adaptive memory system that organizes AI agent memories into a graph and experimentally evaluates structural changes to retrieval.

Team AlphaQ

Nitesh Tanawade | Pranav Tanawade | Pratham Shid

PS 3



Storing information is not enough

As memory grows, fixed structures struggle to deliver what exists.

AI systems accumulate knowledge across projects, preferences, conversations, and documentation.
When memory structures stay fixed, retrieval becomes inefficient and valuable information is missed.

1 Growing memory
Knowledge accumulates across sources and topics.

→

2 Overloaded modules
Fixed structures become crowded and inefficient.

→

3 Missed relevant memories
The right information exists but isn't retrieved.

→

4 Inefficient retrieval
Queries are routed poorly, wasting time and resources.

WHO IS AFFECTED

AI assistants with long-term memory

Agentic applications with growing knowledge bases

Retrieval-enhanced generation systems

Applications that repeatedly access information spread across topics

EVIDENCE FROM MORPHE EXPERIMENTS*

1
Fixed graph routing lowered nDCG@10 versus global dense retrieval by 0.094 on SciFact, 0.039 on NFCorpus and 0.010 on ArguAna (internal partition).

2
Graph routing cut the candidate pool by 77 to 89 percent, but this gave no quality or latency advantage.

3
On SciFact the fixed graph left 66 relevant documents in modules it never searched, across 64 test queries.

* Results of MORPHE's own BEIR experiments, not industry-wide statistics.

PROPOSED SOLUTION

Memory organization as a testable design decision

Graph-structured memory
Memories live in modules linked by a hierarchy and co-access edges.

Graph-guided retrieval
Four strategies compared: global, fixed graph, progressive, soft routing.

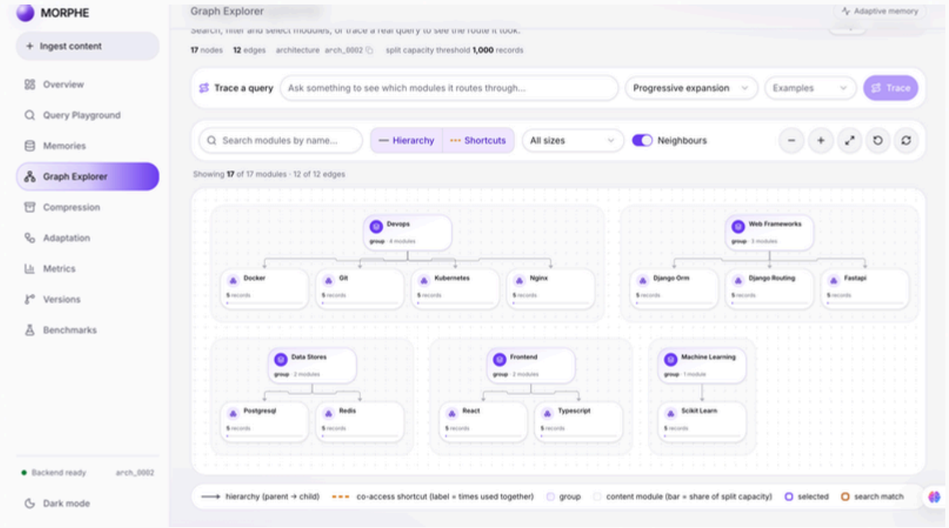
Adaptive restructuring
Split, merge and shortcut changes are proposed from real usage.

Benchmark-validated changes
Candidates are evaluated in a sandbox before any commit.

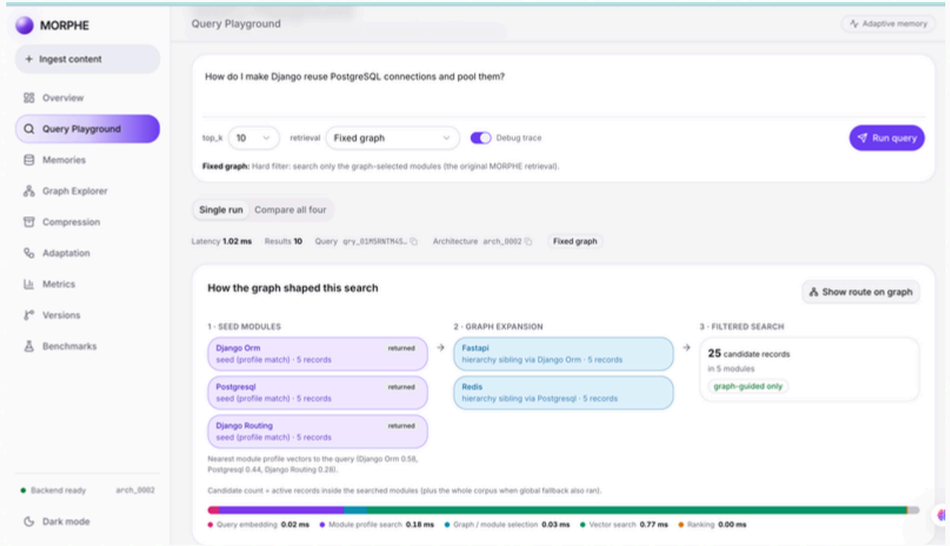
Versioning and rollback
Checksummed architecture versions; any version can be restored.

Observable dashboard
Graph Explorer, Query Playground, Benchmarks and Versions pages.

Reproducible benchmarking
BEIR harness with paired statistics and frozen baselines.



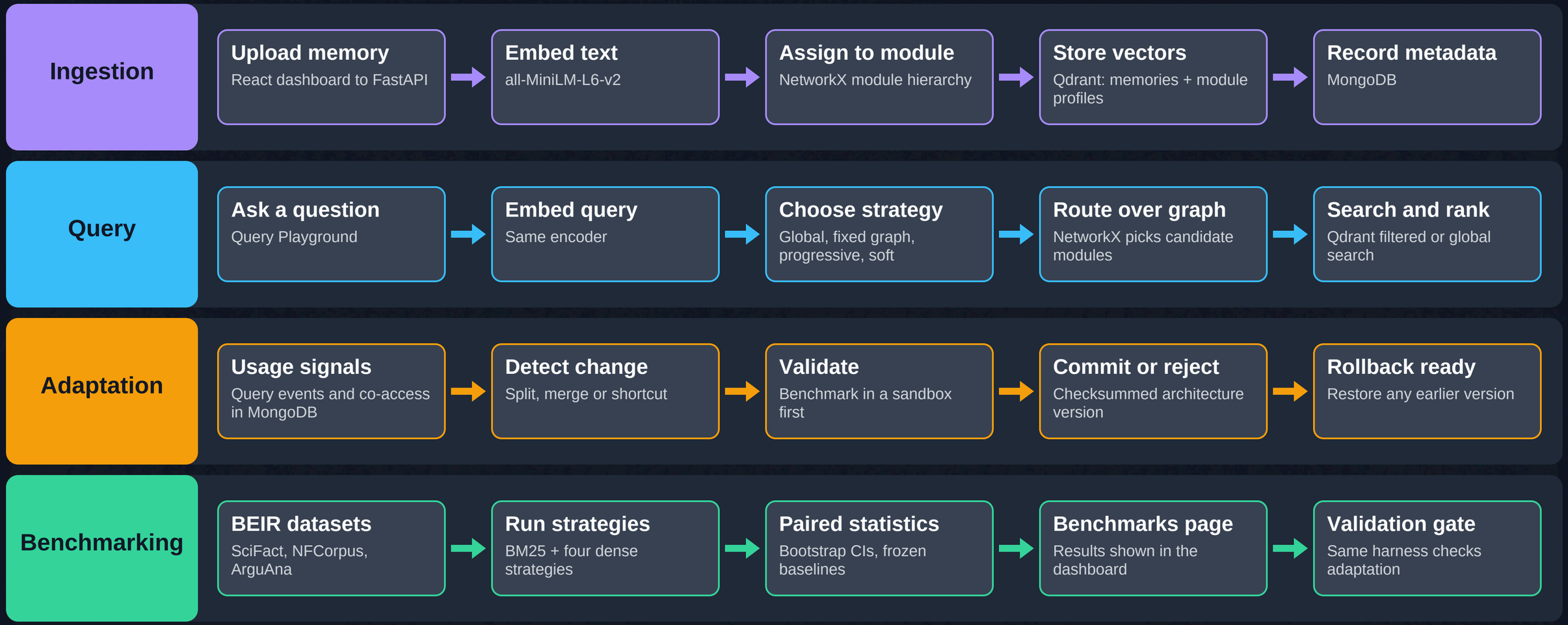
Graph Explorer: module graph laid out from the real hierarchy



Query Playground: fixed-graph retrieval trace on one query

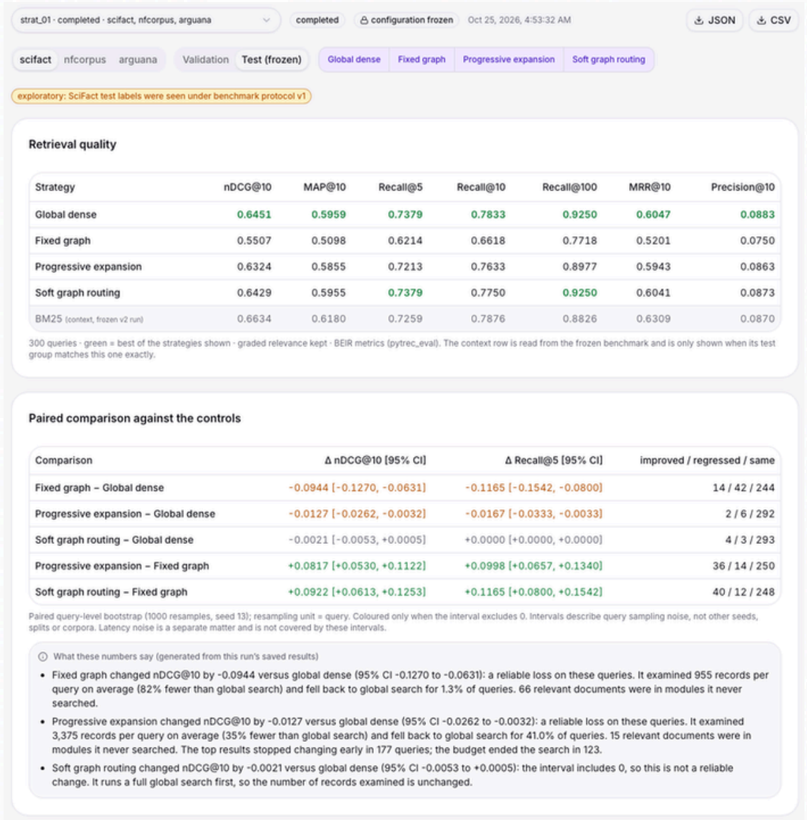
Technical approach and system architecture

Four data flows across React, FastAPI, Qdrant, NetworkX and MongoDB

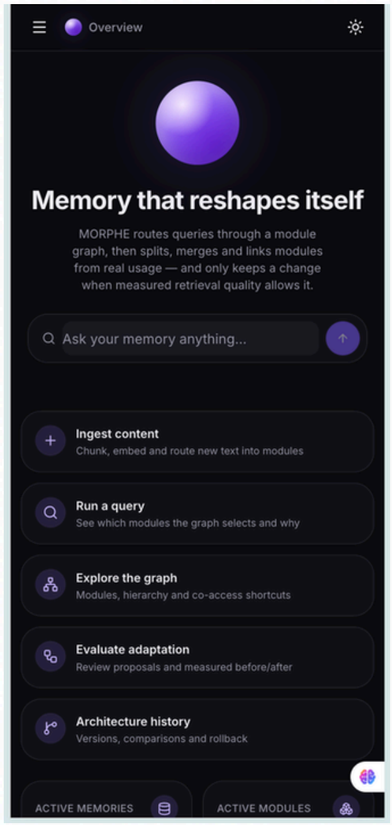


React | TypeScript | Vite | Tailwind CSS | shadcn/ui | React Flow | FastAPI | Pydantic | all-MiniLM-L6-v2 | Qdrant | NetworkX | MongoDB | BEIR

Prototype status, results and risks



Benchmarks > Retrieval strategies, saved run strat_01 (crop of the live app)



Overview, mobile layout

nDCG@10 comparison (test split)

	Global dense	Fixed graph	Progressive	Soft routing
SciFact (exploratory)	0.6451	0.5507	0.6324	0.6429
NFCorpus (confirmatory)	0.3159	0.2771	0.3143	0.3151
ArguAna (internal partition, not an official score)	0.5088	0.4989	0.5081	0.5093

Key conclusion

Graph structure makes memory inspectable and safe to change, but graph-restricted retrieval did not beat global dense search. Fixed graph lowered nDCG@10 on all three datasets (-0.094 / -0.039 / -0.010); progressive and soft routing recover most of it, not more.

Risks and mitigations

Routing misses (progressive expansion and soft routing evaluated)

Retrieval latency (graph and vector-store costs measured separately)

Uncertain adaptation gains (benchmark validation before any structural commit)

Data consistency (versioned state, integrity checks, recovery)

Limited evaluation scale (more datasets, seeds and hardware in future work)

186
Backend passing tests

70
Frontend passing tests

60
End-to-end (Playwright) passing tests

0
Frozen baseline check problems

Impact, future scope and references

EXPECTED IMPACT

More adaptable memory organization for long-running agents

Better inspection of how memory relationships influence retrieval

Evidence-based decisions about graph structure

Recoverable architecture changes

A reproducible framework for comparing retrieval strategies

SCALABILITY DIRECTIONS

Larger datasets and longer agent memory workloads

Multiple embedding models

Global dense versus hybrid versus graph-guided retrieval on larger corpora

Better routing calibration

Find when graph relationships really beat dense search

ROADMAP

1

WORKING PROTOTYPE (DONE)

Ingestion and retrieval
Interactive graph
Benchmark harness
Versioning and recovery

→

2

RESEARCH VALIDATION (NEXT)

Improve routing policies

Evaluate merges and shortcut edges on controlled workloads

Repeated-seed experiments
Larger corpora

→

3

FUTURE SYSTEM (EXPLORE)

Workload-aware memory organization

Efficient retrieval at scale

Operational safeguards if justified

REFERENCES

BEIR
github.com/beir-cellar/beir

Sentence Transformers
sbert.net

Qdrant
qdrant.tech/documentation

NetworkX
networkx.org/documentation

FastAPI
fastapi.tiangolo.com

MORPHE Technical Project Specification v1.0

Thank You